Programiz

C Online Compiler

main.c

Share

Run

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

void deleteAtStart(struct Node **head) {
    if (*head == NULL) {
        printf("List is empty\n");
        return;
    }

    struct Node *temp = *head;
    *head = (*head)->next;
    free(temp);
}

void display(struct Node *head) {
    struct Node *temp = head;

    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }

```

Output

Clear

```
Original List:
10 -> 20 -> 30 -> 40 -> NULL
After deleting first node:
20 -> 30 -> 40 -> NULL


=== Code Execution Successful ===
```

Programiz
C Online Compiler

Programiz PRO
Learn Data Structures and Algorithms with live code visualizations
Try Now

main.c

Share Run

```c
    printf("NULL\n");
}

int main() {
    struct Node *head, *first, *second, *third;

    head = (struct Node*)malloc(sizeof(struct Node));
    first = (struct Node*)malloc(sizeof(struct Node));
    second = (struct Node*)malloc(sizeof(struct Node));
    third = (struct Node*)malloc(sizeof(struct Node));

    head->data = 10;
    head->next = first;

    first->data = 20;
    first->next = second;

    second->data = 30;
    second->next = third;

    third->data = 40;
    third->next = NULL;

    printf("Original List:\n");
    display(head);

    deleteAtStart(&head);
```

Output

Clear

```
Original List:
10 -> 20 -> 30 -> 40 -> NULL
After deleting first node:
20 -> 30 -> 40 -> NULL


=== Code Execution Successful ===
```

Programiz
C Online Compiler

Programiz PRO
Never struggle with DSA again. Learn with interactive visuals
Try Now

main.c

Share Run

```c
    head = (struct Node*)malloc(sizeof(struct Node));
    first = (struct Node*)malloc(sizeof(struct Node));
    second = (struct Node*)malloc(sizeof(struct Node));
    third = (struct Node*)malloc(sizeof(struct Node));

    head->data = 10;
    head->next = first;

    first->data = 20;
    first->next = second;

    second->data = 30;
    second->next = third;

    third->data = 40;
    third->next = NULL;

    printf("Original List:\n");
    display(head);

    deleteAtStart(&head);

    printf("After deleting first node:\n");
    display(head);

    return 0;
}
```

Output

Clear

```
Original List:
10 -> 20 -> 30 -> 40 -> NULL
After deleting first node:
20 -> 30 -> 40 -> NULL


=== Code Execution Successful ===
```